

prone. Once the interface of each component is defined, the code of the classes can even be written by different programmers.

With the MVC approach, each component demonstrates the core OOP concepts of encapsulation and abstraction. Using an MVC object structure, the model can change the way it represents the data internally without affecting the controller or the view. As mentioned previously, the model might contain more data than any single view needs. And as long as the controller doesn't change the way it communicates with the model, and the model continues to return the requested information to the view in an agreed-upon way, the model can add new data without breaking the system.

The MVC model also makes it easy to add enhancements. For example, in our dice-rolling program, the model could keep track of the count of the different combinations of rolls of the two dice that make up each outcome, such as getting a 5 by rolling a 1 and a 4 or a 2 and a 3. We could then modify the `BarChart` view to obtain this additional information from the model and show each bar split into smaller bars to display the percentages of each combination.

Each of the `View` objects is entirely customizable. The `TextView` could use different fonts and font sizes, or a different layout. The `PieView` could show the wedges in different colors. The bars in the `BarView` could be thicker or taller, or be shown in different colors, or even be displayed horizontally. Any such changes would be made only in the appropriate `View` object, completely independent of the model or the controller.

The MVC pattern also makes it easy to add a new way to view the data, by writing a new `View` class. The only additional changes required would be to have the controller draw another radio button, instantiate the new `View` object, and call the new `View` object's `update()` method when the user selects the new view.